`Producer<T>` interface for objects of type `<T>`, and they are used to compose asynchronous, observable data streams: `Mono<T>` and `Flux<T>`:

- `Mono<T>`—Represents a single asynchronous value or empty result (0..1)
- `Flux<T>`—Represents an asynchronous sequence of zero or more items (0..N)

In a Java stream, you would process objects like `Optional<Customer>` or `Collection <Customer>`. In a reactive stream, you would have `Mono<Customer>` or `Flux<Customer>`. The possible outcomes of a reactive stream are an empty result, a value, or an error. All of them are handled as data. When the publisher returns all the data, we say that the reactive stream has been *completed* successfully.

Consumers are called *subscribers* because they subscribe to a publisher and are notified whenever new data is available. As part of the *subscription*, consumers can also define backpressure by informing the publisher that they can process only a certain amount of data at a time. That is a powerful feature that puts consumers in control of how much data is received, preventing them from being overwhelmed and becoming unresponsive. Reactive streams are only activated if there's a subscriber.

You can build reactive streams that combine data from different sources and manipulate it using Reactor's vast collection of *operators*. In a Java stream, you can use a fluent API to process data through operators like `map`, `flatMap`, or `filter`, each of which builds a new `Stream` object that keeps the previous step immutable. Similarly, you can build reactive streams